rqbit FilesystemStorage 구현 분석

개요

FilesystemStorage는 rqbit의 기본 스토리지 구현체로, 실제 파일시스템에 토렌트 데이터를 저장하고 관리합니다. 이 구현은 고성능 I/O, 동시성 제어, 플랫폼별 최적화를 통해 효율적인 파일 관리를 제공합니다.

FilesystemStorage 구조 분석

기본 구조체

```
pub struct FilesystemStorage {
pub(super) output_folder: PathBuf,
pub(super) opened_files: Vec<OpenedFile>,
}
```

주요 필드:

- `output_folder` : 토렌트 파일들이 저장될 기본 디렉토리
- `opened_files` : 열린 파일들의 벡터, 각 파일은 `OpenedFile` 래퍼로 관리

FilesystemStorageFactory

```
#[derive(Default, Clone, Copy)]
```

```
pub struct FilesystemStorageFactory {}
```

특징:

- 상태가 없는 팩토리 (stateless)
- `Default` 구현으로 간단한 생성
- `Clone`, `Copy` 지원으로 효율적인 복사

OpenedFile 래퍼 분석

구조체 정의

```
#[derive(Default, Debug)]
struct OpenedFileLocked {
path: PathBuf,
fd: Option<File>,
#[cfg(windows)]
tried_marking_sparse: bool,
}

#[derive(Debug)]
pub(crate) struct OpenedFile {
file: RwLock<OpenedFileLocked>,
}
```

동시성 제어 메커니즘

읽기/쓰기 잠금

```
pub fn lock_read(&self) -> crate::Result<impl Deref<Target = File>> {
RwLockReadGuard::try_map(self.file.read(), |f| f.as_ref())
.ok()
.ok_or(Error::FsFileIsNone)
}

pub fn lock_write(&self) -> crate::Result<impl DerefMut<Target = File>> {
RwLockWriteGuard::try_map(self.file.write(), |f| f.as_mut())
.ok()
.ok_or(Error::FsFileIsNone)
}
```

특징:

- `RwLock` 을 사용한 다중 읽기, 단일 쓰기 제어
- `try_map` 을 통한 안전한 옵션 언래핑
- 타입 안전성을 위한 `Deref` 트레이트 활용

파일 복제 메커니즘

```
pub fn take_clone(&self) -> anyhow::Result<Self> {
let f = std::mem::take(&mut *self.file.write());
```

```
Ok(Self {
file: RwLock::new(f),
})
}
```

용도:

- 토렌트 일시정지 시 스토리지 교체
- 메모리 효율적인 파일 핸들 이동

플랫폼별 I/O 구현

OurFileExt 트레이트

```
pub(crate) trait OurFileExt {
fn pwrite_all_vectored(&self, offset: u64, bufs: [IoSlice<'_>; 2]) -> anyhow::Re
fn pread_exact(&self, offset: u64, buf: &mut [u8]) -> anyhow::Result<()>;
fn pwrite_all(&self, offset: u64, buf: &[u8]) -> anyhow::Result<()>;
}
```

Unix 구현

```
#[cfg(unix)]
fn pwrite_all_vectored(&self, offset: u64, bufs: [IoSlice<'_>; 2]) -> anyhow::Re
nix::sys::uio::pwritev(self, &bufs, offset.try_into()?).context("error calling p
```

```
}

#[cfg(unix)]
fn pread_exact(&self, offset: u64, buf: &mut [u8]) -> anyhow::Result<()> {
use std::os::unix::fs::FileExt;
Ok(self.read_exact_at(buf, offset)?)
}
```

특징:

- `pwritev` 시스템 콜을 통한 벡터화된 쓰기
- `read_exact_at` / `write_all_at` 을 통한 원자적 I/O
- 파일 포인터 이동 없이 직접 오프셋 접근

Windows 구현

```
#[cfg(windows)]
fn pwrite_all(&self, offset: u64, buf: &[u8]) -> anyhow::Result<()> {
use std::os::windows::fs::FileExt;
let mut remaining = buf.len();
let mut buf = buf;
let mut offset = offset;
while remaining > 0 {
let written = self.seek_write(&buf[..remaining], offset)?;
remaining -= written;
```

```
offset += written as u64;

buf = &buf[written..];

}

Ok(())

}
```

특징:

- 부분 쓰기 처리를 위한 루프 구현
- `seek_write` / `seek_read` 를 통한 오프셋 기반 I/O
- 완전한 데이터 쓰기 보장

벡터화된 I/O 최적화

```
#[cfg(not(unix))]

fn pwrite_all_vectored(&self, offset: u64, bufs: [IoSlice<'_>; 2]) -> anyhow::Re

match (bufs[0].len(), bufs[1].len()) {

(len, 0) if len > 0 => {

self.pwrite_all(offset, &bufs[0])?;

Ok(len)

}

(0, len) if len > 0 => {

self.pwrite_all(offset, &bufs[1])?;

Ok(len)

}
```

```
(0, 0) => Ok(0),

(l0, l1) => {

// 메모리에서 버퍼 연결하여 단일 쓰기 호출

use librqbit_core::constants::CHUNK_SIZE;

let mut buf = [0u8; CHUNK_SIZE as usize];

buf.get_mut(..l0).context("buf too small")?.copy_from_slice(&bufs[0]);

buf.get_mut(l0..l0 + l1).context("buf too small")?.copy_from_slice(&bufs[1]);

self.pwrite_all(offset, &buf[..l0 + l1])?;

Ok(l0 + l1)

}

}

}
```

최적화 전략:

- 단일 버퍼인 경우 직접 쓰기
- 두 버퍼인 경우 메모리에서 연결 후 단일 쓰기
- 시스템 콜 횟수 최소화

Sparse 파일 지원 (Windows)

Sparse 파일 마킹

```
#[cfg(windows)]
```

```
pub fn mark_file_sparse(f: &std::fs::File) -> bool {
use std::os::windows::io::AsRawHandle;
use windows::{
Win32::Foundation::HANDLE, Win32::System::IO::DeviceIoControl,
Win32::System::Ioctl::FSCTL_SET_SPARSE,
};

let handle = HANDLE(f.as_raw_handle());
unsafe { DeviceIoControl(handle, FSCTL_SET_SPARSE, None, 0, None, 0, None, None)
}
```

Sparse 파일 관리

```
#[cfg(windows)]
pub fn try_mark_sparse(&self) -> crate::Result<impl Deref<Target = File>> {
{
let g = self.file.read();
if g.tried_marking_sparse {
return RwLockReadGuard::try_map(g, |f| f.fd.as_ref())
.ok()
.ok_or(Error::FsFileIsNone);
}
}
```

```
let mut g = self.file.write();

if !g.tried_marking_sparse {

g.tried_marking_sparse = true;

let f = g.fd.as_ref().ok_or(Error::FsFileIsNone)?;

tracing::debug!(path=?g.path, marked=super::sparse::mark_file_sparse(&f), "marki

}

let g = parking_lot::RwLockWriteGuard::downgrade(g);

Ok(RwLockReadGuard::try_map(g, |f| f.fd.as_ref()).ok().unwrap())

}
```

특징:

- 한 번만 sparse 마킹 시도 (`tried_marking_sparse` 플래그)
- 디스크 공간 효율성 향상
- 대용량 파일의 빠른 생성

메모리 매핑 구현 (MmapFilesystemStorage)

구조체 정의

```
pub struct MmapFilesystemStorage {

opened_mmaps: Vec<OpenedMmap>,

fs: FilesystemStorage,

}
```

```
type OpenedMmap = RwLock<MmapMut>;
```

초기화 과정

```
fn init(&mut self, shared: &ManagedTorrentShared, metadata: &TorrentMetadata) ->
self.fs.init(shared, metadata)?;
let mut mmaps = Vec::new();
for (idx, file) in self.fs.opened_files.iter().enumerate() {
let fg = file.lock_write()?;
fg.set_len(metadata.file_infos[idx].len).context("mmap storage: error setting le
let mmap = unsafe { MmapOptions::new().map_mut(&*fg) }.context("error mapping fi
mmaps.push(RwLock::new(mmap));
}
self.opened_mmaps = mmaps;
Ok(())
}
```

과정:

1. 기본 파일시스템 스토리지 초기화
2. 각 파일의 크기 설정
3. 메모리 매핑 생성
4. RwLock으로 동시성 제어

메모리 매핑 I/O

```
fn pread_exact(&self, file_id: usize, offset: u64, buf: &mut [u8]) -> anyhow::Re
let g = self.opened_mmaps.get(file_id).context("no such file")?.read();
let start = offset;
let end = offset + buf.len() as u64;
let start = start.try_into()?;
let end = end.try_into()?;
buf.copy_from_slice(g.get(start..end).context("bug")?);
Ok(())
}
```

장점:

- OS 페이지 캐시 활용
- 시스템 콜 오버헤드 제거
- 대용량 파일 효율적 처리

파일 작업 통합 (FileOps)

구조체 정의

```
pub(crate) struct FileOps<'a> {
torrent: &'a ValidatedTorrentMetaV1Info<ByteBufOwned>,
files: &'a dyn TorrentStorage,
```

```
file_infos: &'a FileInfos,

phantom_data: PhantomData<Sha1>,

}
```

조각 검증 과정

```
pub fn check_piece(&self, piece_index: ValidPieceIndex) -> anyhow::Result<bool>

let mut h = Sha1::new();

let piece_length = self.torrent.lengths().piece_length(piece_index);

let mut absolute_offset = self.torrent.lengths().piece_offset(piece_index);

let mut buf = vec![0u8; std::cmp::min(65536, piece_length as usize)];


// 여러 파일에 걸친 조각 읽기 및 해시 계산

for (file_idx, fi) in self.file_infos.iter().enumerate() {

// 파일별 읽기 로직

update_hash_from_file(file_idx, fi, absolute_offset, self.files, &mut h, &mut bu

}


// 해시 비교

self.torrent.info().compare_hash(piece_index.get(), h.finish())

}
```

청크 쓰기 과정

```
pub fn write_chunk(&self, who_sent: PeerHandle, data: &Piece<ByteBuf<'a>>, chunk

let mut absolute_offset = self.torrent.lengths().chunk_absolute_offset(chunk_inf

let mut data = DoubleBufHelper::new(data.data().0, data.data().1);



for (file_idx, file_info) in self.file_infos.iter().enumerate() {

// 파일 경계 계산

let remaining_len = file_len - absolute_offset;

let to_write = std::cmp::min(data.len() as u64, remaining_len).try_into()?;



// 벡터화된 쓰기

let slices = data.as_ioslices(to_write);

if !file_info.attrs.padding {

let written = self.files.pwrite_all_vectored(file_idx, absolute_offset, slices)?

}

data.advance(to_write);

}

Ok(())

}
```

초기화 및 파일 관리

파일 생성 과정

```
fn init(&mut self, shared: &ManagedTorrentShared, metadata: &TorrentMetadata) ->

let mut files = Vec::<OpenedFile>::new();

for file_details in metadata.file_infos.iter() {

let mut full_path = self.output_folder.clone();

let relative_path = &file_details.relative_filename;

full_path.push(relative_path);



if file_details.attrs.padding {

files.push(OpenedFile::new_dummy());

continue;

};



// 디렉토리 생성

std::fs::create_dir_all(full_path.parent().context("bug: no parent")?)?;

// 파일 생성/열기

let f = if shared.options.allow_overwrite {

OpenOptions::new()

.create(true)

.truncate(false)

.read(true)

.write(true)

.open(&full_path)?
```

```
} else {
// 새 파일만 생성 (덮어쓰기 방지)
OpenOptions::new().create_new(true).write(true).open(&full_path)?;
OpenOptions::new().read(true).write(true).open(&full_path)?
};
files.push(OpenedFile::new(full_path.clone(), f));
}
self.opened_files = files;
Ok(())
}
```

특징:

- 패딩 파일은 더미로 처리
- 디렉토리 자동 생성
- 덮어쓰기 옵션 지원
- 안전한 파일 생성

파일 제거 및 정리

```
fn remove_directory_if_empty(&self, path: &Path) -> anyhow::Result<()> {
let path = self.output_folder.join(path);
if !path.is_dir() {
anyhow::bail!("cannot remove dir: {path:?} is not a directory")
}
```

```
if std::fs::read_dir(&path)?.count() == 0 {
std::fs::remove_dir(&path).with_context(|| format!("error removing {path:?}"))
} else {
warn!("did not remove {path:?} as it was not empty");
Ok(())
}
}
```

안전성:

- 빈 디렉토리만 제거
- 경고 로그를 통한 상태 알림
- 오류 컨텍스트 제공

성능 최적화 기법

1. 벡터화된 I/O

- `pwritev` 시스템 콜 활용 (Unix)
- 메모리 연결을 통한 단일 쓰기 (Windows)
- I/O 시스템 콜 횟수 최소화

2. 메모리 매핑

- OS 페이지 캐시 활용
- 대용량 파일 효율적 처리
- 시스템 콜 오버헤드 제거

3. Sparse 파일 지원

- 디스크 공간 효율성
- 빠른 대용량 파일 생성
- Windows 플랫폼 최적화

4. 동시성 최적화

- RwLock을 통한 다중 읽기 지원
- 파일별 독립적 잠금
- 데드락 방지 설계

오류 처리 및 복구

오류 타입

- I/O 오류: 파일 읽기/쓰기 실패
- 권한 오류: 파일 접근 권한 부족
- 공간 오류: 디스크 공간 부족

복구 전략

- 컨텍스트 정보를 포함한 오류 전파
- 파일별 오류 격리
- 안전한 리소스 정리

결론

FilesystemStorage 구현은 다음과 같은 특징을 제공합니다:

1. **플랫폼 최적화**: Unix/Windows별 최적화된 I/O

2. **고성능**: 벡터화된 I/O와 메모리 매핑 지원

3. **동시성**: RwLock을 통한 안전한 다중 접근

4. **효율성**: Sparse 파일과 캐싱 지원

5. **안전성**: 포괄적인 오류 처리와 리소스 관리

이러한 구현을 통해 rqbit은 다양한 환경에서 효율적이고 안정적인 파일 저장 기능을 제공합니다.